

Promotional Demand Forecasting

Technical Approach Document

Saim Malik

Prepared for Akuret Solutions
CTO: Kevin DiGilio

Problem	Predict demand uplift from promotions 1-2 weeks ahead
Data	Transactional sales data + promotion calendar
Core Task	Time series forecasting (regression), NOT classification
Approach	Classical models -> ML -> Agent automation
Key Models	Prophet, XGBoost/LightGBM, DeepAR, TFT
Agent	LangGraph/LangChain for automated weekly pipeline

Table of Contents

1. The Business Problem

Why this is forecasting, not classification

2. Types of Time Series Problems

Univariate, multivariate, probabilistic, hierarchical

3. The Data

Transactional data structure, additional sources, quality issues

4. Feature Engineering

Time, lag, promotion, and cross-product features

5. Model Selection Framework

Decision tree for choosing the right model

6. Tier 1: Classical Models

Prophet, SARIMA, ETS — with pros/cons/code

7. Tier 2: ML Models

XGBoost, LightGBM, Random Forest

8. Tier 3: Deep Learning

LSTM, DeepAR, TFT — state-of-the-art

9. Model Comparison Matrix

9 models compared on 7 dimensions

10. Clustering as Preprocessing

Store, product, and customer segmentation

11. Evaluation Metrics

MAE, MAPE, RMSE, bias — NOT accuracy/recall

12. The Agent Architecture

Automated weekly forecasting pipeline

13. Complete Pipeline

End-to-end architecture diagram

14. Databases and Tools

TimescaleDB, libraries, feature stores

15. Key Concepts

Stationarity, elasticity, cannibalization, reconciliation

16. Common Pitfalls

What goes wrong and how to avoid it

17. Recommended Approach

Phased implementation for Akuret

18. Interview Q&A;

6 questions with polished answers

1. The Business Problem

A grocery retailer announces a promotion (e.g., BOGO on Coca-Cola starting in 2 weeks). They need to predict: **how many extra units should we stock?** Order too few = stockouts and lost sales. Order too many = waste and tied-up capital.

This is a **demand forecasting** problem, specifically **promotional demand forecasting** - predicting the sales uplift caused by a promotion above the baseline demand.

Why this is NOT classification

Problem Type	What it predicts	Example
Classification	A category/label	"Will customer churn? Yes/No"
Regression	A continuous number	"How many units will sell?"
Forecasting	A number over time	"Daily sales for the next 14 days"
Clustering	Groups of similar items	"Which stores behave similarly?" (preprocessing)

Akuret's problem = Forecasting (a type of regression over time). The output is a predicted number (units to sell per SKU per store per day), not a label or category.

The math behind promotional lift

Promotional Demand = Baseline Demand $\times$ (1 + Lift Factor)

Example: Baseline 100 units/week, BOGO lift = 2.5x, Predicted = 250 units, Extra to order = 150

2. Types of Time Series Problems

Type	Description	Models
Univariate	Predict from single variable history	ARIMA, ETS, Prophet
Multivariate	Predict using multiple variables	XGBoost, TFT, LSTM
Probabilistic	Predict a distribution, not a point	DeepAR, Prophet
Hierarchical	Forecasts at multiple levels must reconcile	MinT, bottom-up
Intermittent	Products that sell sporadically	Croston's method

*Akuret's problem is primarily **multivariate + probabilistic** - multiple input features producing a demand prediction with confidence intervals.*

Stationarity

A time series is **stationary** if its statistical properties (mean, variance) don't change over time. Most models assume or require stationarity. Non-stationary patterns: **Trend** (steady increase/decrease), **Seasonality** (weekly/monthly/yearly cycles), **Structural breaks** (sudden permanent shifts). Fix with: differencing, seasonal differencing, log transformation. Test with: ADF (Augmented Dickey-Fuller) test.

Decomposition

Time Series = Trend + Seasonality + Residual (additive)

The promotion creates a **spike** in the residual. Our job is to predict the size of that spike.

3. The Data - What We're Working With

Transactional data (the spreadsheet Kevin mentioned)

Field	Description	Example
date	Transaction date	2024-03-15
store_id	Which store	STORE_042
sku	Product identifier	COKE_12OZ_REG
units_sold	Quantity sold	48
unit_price / regular_price	Selling and normal price	\$1.20 / \$1.50
promo_flag / promo_type	Is promo active? What type?	1 / BOGO
promo_depth	Discount percentage	50%

Additional data sources

Source	What it adds	Why it matters
Calendar	Holidays, pay days	Black Friday spikes, weekend patterns
Weather	Temperature, rain	Ice cream in heat, soup in cold
Competitor	Competitor promos	Pepsi promo cannibalizes Coke baseline
Store attributes	Location, size, demographics	Urban vs rural promo response
Inventory	Stock levels, lead times	Constrain forecast to what is orderable

Data quality issues to expect

Missing data - stores didn't report some days. **Censored demand** - out-of-stock days show sales=0 but actual demand was higher (stockout bias). **Cannibalization** - Coke promo steals Pepsi sales. **Halo effect** - Coke promo increases chip sales. **Forward buying** - customers stockpile during promo, post-promo sales drop.

4. Feature Engineering - The Real Work

Time-based features

day_of_week, week_of_year, month, is_holiday, days_to_holiday, is_payday, is_weekend

Lag features (historical patterns)

Feature	What it captures
sales_lag_7, sales_lag_14	Same day last week / 2 weeks ago
sales_lag_365	Same day last year (yearly seasonality)
rolling_mean_7, rolling_mean_28	Short-term and monthly trend
rolling_std_7	Recent volatility / demand variability

Promotion features (the critical ones)

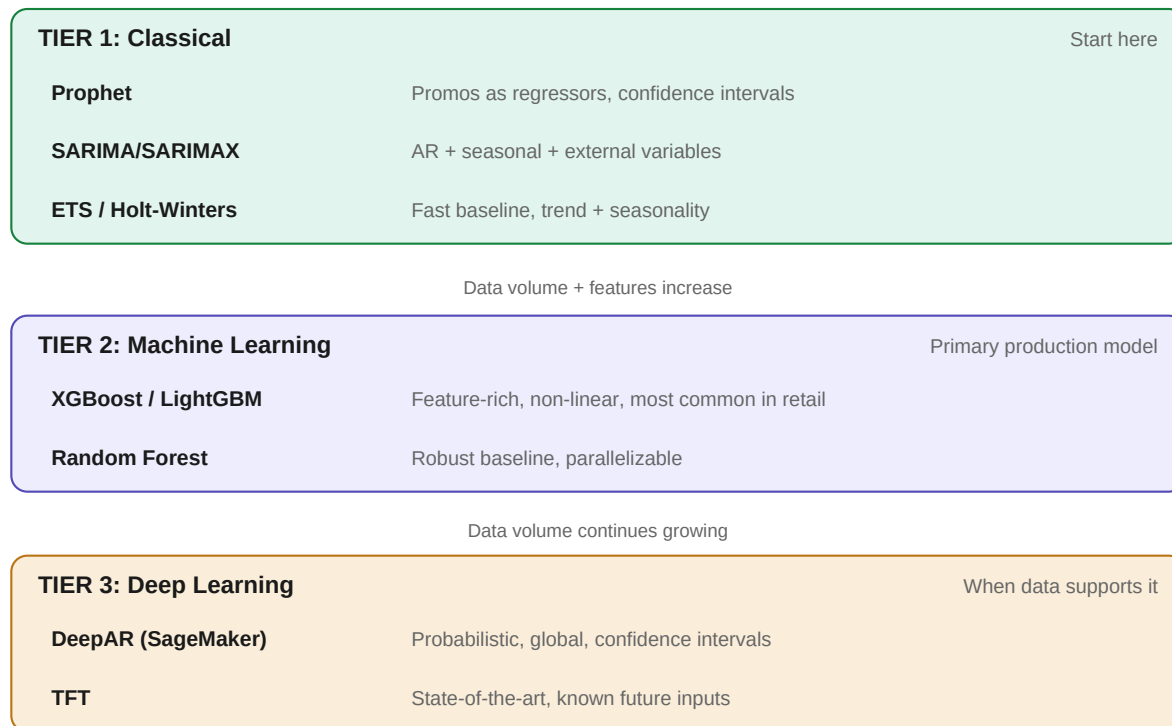
Feature	What it captures
is_promo, promo_type_encoded	Binary flag + one-hot type (BOGO, PCT_OFF, etc.)
promo_depth, promo_duration	Discount % and length of promotion
days_into_promo	Day 1 vs day 7 (fatigue effect)
days_since_last_promo	Recency of last promotion on this product
historical_lift	Average lift from past promos of same type
competitor_promo	Is competitor running similar promo?
cannibalization_index	Historical % of sales stolen from substitutes

Feature engineering matters more than model complexity. A well-featured XGBoost will beat a poorly-featured TFT.

5. Model Selection Framework

The selection depends on three factors: **data volume** (how many data points per series), **number of features** (univariate vs multivariate), and **number of time series** (single product vs thousands of SKU/store combinations).

Model Selection: Three Tiers



6. Tier 1: Classical Statistical Models

Prophet (Meta)

Decomposable model: $y(t) = \text{trend} + \text{seasonality} + \text{holidays} + \text{regressors} + \text{error}$. The regressors term lets you add promotions as external inputs. Handles missing data, outliers, and multiple seasonalities natively. Gives confidence intervals.

```
model = Prophet()
model.add_regressor('is_promo')
model.add_regressor('promo_depth')
model.fit(df)
forecast = model.predict(future_df) # future_df includes known promo schedule
```

Strengths	Weaknesses
Multiple seasonalities, robust to missing data	Linear regressor effects only
Can add promos as regressors	Can't model complex interactions
Built-in confidence intervals	One model per series (no cross-learning)
Minimal tuning needed	Can overfit short series

SARIMA / SARIMAX

AutoRegressive Integrated Moving Average with Seasonal terms. SARIMAX adds exogenous variables (promotions). Parameters: SARIMA(p,d,q)(P,D,Q,s) where p=AR order, d=differencing, q=MA order, s=seasonal period. Well-understood, interpretable, fast. But univariate (SARIMA) and requires manual parameter tuning.

ETS / Holt-Winters

Exponential Smoothing giving higher weight to recent data. Three variants: Simple (level), Holt (level+trend), Holt-Winters (level+trend+seasonality). Blazing fast - can run across 10,000+ SKUs in seconds. But univariate only, no external features, point forecasts only.

***When to use for Akuret:** Prophet as the first production model (handles promos natively). SARIMA and ETS as quick baselines to benchmark against.*

7. Tier 2: Machine Learning Models

XGBoost / LightGBM (Gradient Boosted Trees)

The most common production model for retail demand forecasting. Transform time series into a supervised learning problem: each row = [lag_1, lag_7, day_of_week, is_promo, promo_depth, ...] -> units_sold. Trees naturally capture non-linear relationships and feature interactions (e.g., 'BOGO on Coke in summer in urban stores').

```
features = ['sales_lag_7', 'rolling_mean_7', 'day_of_week',
            'is_promo', 'promo_type', 'promo_depth', 'historical_lift']
model = lgb.LGBMRegressor(n_estimators=1000, learning_rate=0.05)
model.fit(X_train[features], y_train)
```


Strengths	Weaknesses
Handles dozens of features naturally	Requires manual feature engineering
Captures non-linear interactions	Can't extrapolate beyond training range
Feature importance (interpretable)	Doesn't inherently understand time ordering
Fast training and inference	Risk of data leakage with lag features

This is likely the primary production model for Akuret. Feature engineering is the hard part; the model itself is proven across retail.

8. Tier 3: Deep Learning Models

DeepAR (Amazon)

Probabilistic, autoregressive forecasting. Trains one **global model across ALL time series** - learns from high-volume products and transfers to low-volume ones. Outputs prediction intervals, not just points: 'Coke will sell 200-280 units (90% confidence)'. Available in **Amazon SageMaker** (your experience!).

Strengths	Weaknesses
Probabilistic - confidence intervals	Needs substantial data across many series
Global model - cross-series learning	Black box - hard to explain predictions
Handles cold-start (new products)	Compute-intensive training
SageMaker native (your experience!)	AWS infrastructure dependency

Temporal Fusion Transformer (TFT)

Google's transformer-based architecture - **state-of-the-art** for this exact problem. Separates inputs into: **Static covariates** (store location, product category), **Known future inputs** (planned promotions, holidays - you KNOW these ahead of time), and **Observed past inputs** (historical sales). Built-in attention-based interpretability shows which features drove each prediction.

Strengths	Weaknesses
State-of-the-art accuracy	Complex - harder to implement and debug
Known future inputs native (perfect for promos!)	Needs 100K+ samples across series
Attention-based interpretability	Computationally expensive training
Probabilistic quantile forecasts	Less production-tested than XGBoost

Aspirational target: Start with XGBoost, graduate to TFT when data volume justifies it.

9. Model Comparison Matrix

Model	Handles Promos	Multi-series	Probabilistic	Data Needed	Speed
ARIMA/SARIMA	No	No	Limited	Small	Fast
ETS	No	No	No	Small	Very fast
Prophet	Yes	No	Yes	Small-Med	Medium
XGBoost	Yes	Yes	No*	Medium	Fast
LSTM/GRU	Yes	Yes	No*	Large	Slow
DeepAR	Yes	Yes	Yes	Large	Slow
TFT	Yes	Yes	Yes	Very Large	Slow

*Can be made probabilistic with quantile regression variants

10. Clustering as Preprocessing

Not all stores respond to promotions the same way. Clustering groups similar entities so we can train separate models or use cluster membership as a feature.

Clustering Type	Features	Algorithms	Result
Store clustering	Revenue, size, location, demographics	K-Means, DBSCAN	"High-traffic urban" vs "Rural low-traffic"
Product clustering	Price, category, elasticity	K-Means, Hierarchical	Products with similar promo response
Customer (RFM)	Recency, frequency, monetary	K-Means, DBSCAN	Which segments respond to promos

Clustering is a **preprocessing step**, not the core model. The forecasting model (XGBoost/TFT) is the core.

11. Evaluation Metrics for Forecasting

Forecasting vs Classification Metrics

DON'T USE (Classification)

- X Accuracy
- X Recall
- X Precision
- X F1 Score

USE THESE (Forecasting)

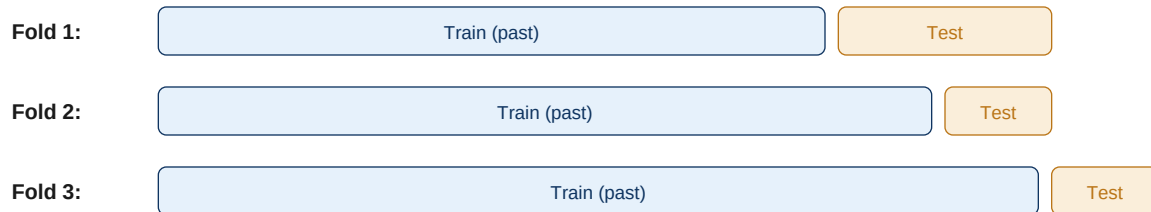
- MAE — avg error in units
- MAPE — avg % error
- RMSE — penalizes big misses
- Forecast Bias — over/under

Key insight: for grocery, slightly over-predicting is cheaper than stockouts during a promo

Metric	Formula	Use case
MAE	$\text{mean}(\text{actual} - \text{predicted})$	Average error in units
MAPE	$\text{mean}(\text{actual} - \text{predicted} / \text{actual}) \times 100$	Average % error across products
RMSE	$\sqrt{\text{mean}((\text{actual} - \text{predicted})^2)}$	Penalizes large errors heavily
WAPE	$\text{sum}(\text{errors}) / \text{sum}(\text{actuals})$	Better than MAPE for low-volume items
Bias	$\text{mean}(\text{predicted} - \text{actual}) / \text{mean}(\text{actual})$	Systematic over/under prediction

Walk-forward validation

Walk-Forward Validation (Correct for Time Series)

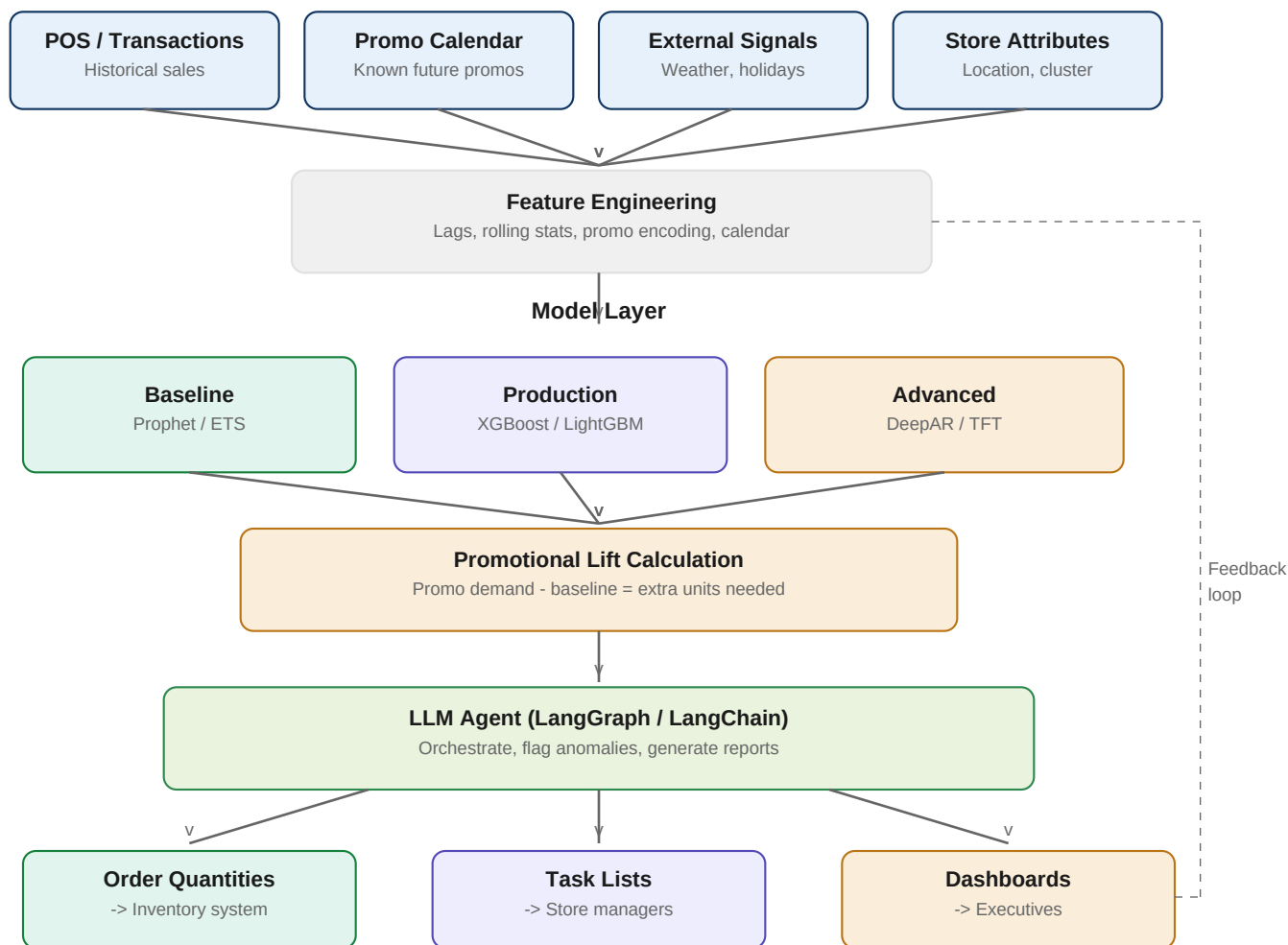


WRONG: Random 80/20 split leaks future data into training!

12-13. The Agent Architecture and Complete Pipeline

The agent automates the entire weekly forecasting cycle. Classical/ML models do the math. The LLM agent does the orchestration. Humans review edge cases only.

End-to-End Forecasting Pipeline



Classical models for prediction. LLM agent for orchestration and reporting. This boundary is critical.

14. Time Series Databases and Tools

Databases

Database	Type	Best for
TimescaleDB	PostgreSQL extension	Time series on Postgres (you know Postgres!)
InfluxDB	Purpose-built TSDB	High-volume metrics, good query language
Amazon Timestream	Managed AWS TSDB	AWS-native, integrates with SageMaker
ClickHouse	Columnar OLAP	Fast analytical queries on large datasets

ML / Forecasting libraries

Library	Key models	Notes
statsmodels	ARIMA, SARIMA, ETS	Classical statistics, well-tested
Prophet	Prophet	Easy, handles promos as regressors
LightGBM / XGBoost	Gradient boosted trees	Primary production model
PyTorch Forecasting	TFT, N-BEATS, DeepAR	Deep learning forecasting
GluonTS	DeepAR, WaveNet	Amazon's forecasting library
Darts	All of the above	Unified API for many models
tsfresh	Feature extraction	Automatic time series features

Orchestration: LangChain/LangGraph (agents), Airflow/Prefect (scheduling), Celery+Redis (async tasks - you already use this)

15. Key Concepts

Concept	What it means	Why it matters
Autocorrelation	Correlation with lagged self	Determines lag features and ARIMA params
Stationarity	Stats properties stable over time	Most models require it, test with ADF
Exogenous variables	External factors (promos, weather)	SARIMAX, Prophet, XGBoost handle these
Promotional elasticity	% demand change per % price change	High-elasticity products benefit most from promos
Cannibalization	Promo steals from substitutes	Must model cross-product effects
Halo effect	Promo lifts complementary products	Coke promo increases chip sales
Forward buying	Customers stockpile during promo	Post-promo sales dip must be predicted
Forecast reconciliation	Hierarchy levels must be consistent	Store forecasts must sum to regional total

16. Common Pitfalls

Pitfall	What goes wrong	How to avoid
Classification metrics	Accuracy/recall for a forecasting problem	Use MAE, MAPE, RMSE, bias
Random train/test split	Leaks future data into training	Walk-forward validation only
Ignoring stockout bias	Zero-sales treated as zero demand	Flag stockout periods, impute demand
Ignoring cannibalization	Each product forecast independently	Model cross-product effects
Point forecasts only	No decision boundary for ordering	Use probabilistic models (DeepAR, TFT)
Same model for all	High-volume and niche products differ	Cluster products, separate models
No production monitoring	Model degrades silently	Track MAPE, bias, data drift weekly

17. Recommended Approach for Akuret

Phase	Timeline	What to do
Phase 1: Baseline	Week 1-2	Data exploration, decomposition, Prophet + ETS baseline, establish metrics
Phase 2: ML	Week 3-4	Feature pipeline, store/product clustering, XGBoost/LightGBM, compare vs baseline
Phase 3: Production	Month 2	Deploy best model (FastAPI + Docker), build LangGraph agent, monitoring dashboard
Phase 4: Advanced	Month 3+	DeepAR on SageMaker, TFT evaluation, cannibalization modeling, feedback loop

18. Interview Q&A; - Forecasting

Q: How would you approach the promotional forecasting problem?

Start with data understanding, establish a baseline with Prophet (handles seasonality, lets me add promos as regressors), then build a feature-rich XGBoost/LightGBM model with lag features, rolling statistics, and promo-specific features. Feature engineering matters more than model complexity. Once the model beats baseline, wrap it in an agent for automated weekly forecasting.

Q: How do you evaluate a forecasting model?

Walk-forward validation - train on past, test on future, slide forward. Never random splits (leaks future data). Metrics: MAPE for accuracy, forecast bias for operational impact. Bias matters more - under-predicting during a promo means stockouts, which is more expensive than slightly over-ordering.

Q: Pretrained model or train from scratch?

Depends on data. With a year+ of weekly data across many SKUs, I'd train XGBoost from scratch - the model needs this retailer's specific patterns. For DeepAR, pretrained weights provide a warm start but fine-tuning on actual data is still necessary. I can only answer definitively after seeing the dataset.

Q: Why not use an LLM for the forecasting?

LLMs can't reliably predict that Coke will sell 247 units next Tuesday. That requires statistical/ML models trained on numerical patterns. The LLM adds value in the agent layer - orchestrating the pipeline, generating reports, explaining anomalies, and providing a conversational interface for managers.

Q: How do you handle the 1-2 week constraint?

It works in our favor. The promotion schedule is a known future input - we know what promo, when, and at what discount. Models like Prophet (regressors), XGBoost (features), and TFT (known future inputs) handle this natively.

Q: How does clustering fit in?

Clustering is preprocessing, not the core model. I'd cluster stores by promo response behavior (high-traffic urban vs rural), then either train separate models per cluster or use cluster membership as a feature. Same for products - group by promotional elasticity so products with similar discount responses share model parameters.

Key correction: This is a forecasting/regression problem, not classification. The output is a predicted number (units to sell), not a category label. Clustering supports the forecasting but is not the core task.